Xling: A Learned Filter Framework for Accelerating High-Dimensional Approximate Similarity Join

1st Yifan Wang
University of Florida
wangyifan@ufl.edu

2nd Vyom Pathak
University of Florida
v.pathak@ufl.edu

3rd Daisy Zhe Wang
University of Florida
daisyw@ufl.edu

arXiv:2402.13397v1 [cs.DB] 20 Feb 2024

***Abstract*—Similarity join is a critical and widely used operation in multi-dimensional data applications, which finds all pairs of close points within a given distance threshold. Being studied for decades, many similarity join methods have been proposed, but they are usually not efficient on high-dimensional space due to the curse of dimensionality and data-unawareness. Inspired by the Bloom join in RDBMS, we investigate the possibility of using metric space Bloom filter (MSBF), a family of data structures checking if a query point has neighbors in a multi-dimensional space, to speed up similarity join. However, there are several challenges when applying MSBF to similarity join, including excessive information loss, data-unawareness and hard constraint on the distance metric, because of which few works are designed in this way.**

In this paper, we propose Xling, a generic framework to build a learning-based metric space filter with any existing regression model, aiming at accurately predicting whether a query point has enough number of neighbors. The framework provides a suite of optimization strategies to further improve the prediction quality based on the learning model, which has demonstrated significantly higher prediction quality than existing MSBF. We also propose XJoin, one of the first filter-based similarity join methods, based on Xling. By predicting and skipping those queries without enough neighbors, XJoin can effectively reduce unnecessary neighbor searching and therefore it achieves a remarkable acceleration. Benefiting from the generalization capability of deep learning models, XJoin can be easily transferred onto new dataset (in similar distribution) without re-training. Furthermore, Xling is not limited to being applied in XJoin, instead, it acts as a flexible plugin that can be inserted to any loop-based similarity join methods for a speedup. Our evaluation shows that Xling not only leads to the high performance of XJoin (e.g., being up to 17x faster than the baselines while maintaining a high quality), but also be able to further speed up many existing similarity join methods with quality guarantee.

***Index Terms*—similarity join, high-dimensional data management, machine learning**

I. Introduction

In multi-dimensional data management, metric space range search (shortly called *range search*) and similarity join are critical operators. Given a distance threshold ϵ, the former operation finds all points in a dataset D whose distance to a given query point is less than ϵ, while the latter operation finds all pairs of points between two datasets R and S whose distance is less than ϵ. As range search can be seen as a special case of similarity join where $|R| = 1$ or $|S| = 1$, unless it is necessary, we will only discuss similarity join in this paper. With the emergence of deep learning, high-dimensional neural embedding has been widely adopted as the data representation in a wide range of applications, which raises the demands for effective and efficient similarity join over high-dimensional data. Those applications include near-duplicate detection [1], [2], [3], [4], [5], data integration [6], [7], [8], data exploration [9], [10], [11], privacy [12], [13] and so on. Specifically, distance between embeddings reflects the semantic similarity between the data objects, meaning that the applications have to frequently search for close points in the embedding space, i.e., the similarity join. In many real-world use cases, the similarity join is approximate, i.e., they do not require 100% accurate results, but do demand high processing speed, especially on large-scale data. A typical example is near-duplicate video retrieval [3] that identifies online videos with identical or almost identical content during the search process to diversify the video search results, in which case missing a few pairs of similar videos is acceptable, while fast response is required.

Existing work on efficient similarity join mainly includes two categories: space-grid [14], [15], [16], [17] and locality-sensitive hashing (LSH) based methods [18], [19], [4], [5], [12], [13]. The former splits the data space into grids and joins the points within the same or neighboring grids, while the latter is essentially an adoption of RDBMS hash-join principle onto the high-dimensional join, i.e., hashing one dataset by LSH and then probing it for the points in another dataset. However, the grid-based methods poorly perform in very high-dimensional space due to the curse of dimensionality, while the unawareness for data distribution usually causes a non-trivial performance degradation to LSH-based methods on unevenly distributed data [20]. In addition, the grid-based methods are usually exact while LSH-based methods are approximate, i.e., their accuracies are respectively 100% and lower than 100%. In this paper we focus on approximate similarity join.

There is one more possible way that speeds up similarity join with *metric space Bloom filter*, inspired by the Bloom join in RDBMS (which uses a Bloom filter to prune unnecessary probings). Metric space Bloom filter (MSBF) is designed for checking whether a query point has neighbors within the given distance threshold. Among the various MSBFs [21], [22], [23], [24], [25], [26], [27], Locality-Sensitive Bloom Filter (LSBF) [21] is the most widely used, based on which a substantial number of MSBFs have been developed. Mirroring to the hashing functions in Bloom filter, LSBF utilizes LSH functions to map a multi-dimensional point to several bits

in the bit array, and determines the neighbor existence by counting the non-zero bits. Given datasets *R* and *S*, an LSBF can be built on *R* and each point $s \in S$ will act as a query. By skipping the range search for those negative queries (i.e., the queries having no neighbors in R), the similarity join can be accelerated. Note that unlike Bloom filter, most MSBFs cannot guarantee a zero false negative rate since they are based on LSH which raises both false positives and negatives.

According to our evaluation, negative queries usually take up a non-trivial portion ($20\% \sim 90\%$), meaning that the filtering should result in a significant performance improvement. But few methods are designed in such a way, due to several problems of MSBF: (1) Given the fact that they are built on top of LSH, their effectiveness is also limited by the data-unawareness. (2) They lose more information compared to the original LSH because of further mapping LSH values to one-dimension bit array, which additionally lowers their effectiveness. (3) They indicate the index of target bit by the LSH value, and this disables them to support many popular distance metrics, like cosine distance where the LSH values are always 0 or 1.

Inspired by the "learned Bloom filter" [28] that enhances Bloom filter with machine learning, we propose a new type of MSBF based on machine learning techniques which addresses the problems above. Specifically, in this paper, we propose **Xling**, a generic framework for building a MSBF with deep learning model. Instead of LSH, Xling relies on the *learned cardinality estimation* techniques which utilize regression models to predict the number of neighbors for range search before actually executing it. Xling is designed to be generic such that any cardinality estimator (or simply, regression model) can be encapsulated into an effective MSBF. Its core is a cardinality estimator with a *Xling decision threshold* (XDT). For a range query, Xling predicts the result cardinality, then determines the query is positive (i.e., having enough neighbors) if the prediction exceeds XDT, otherwise it is negative (i.e., being without enough neighbors). Note that we mention "enough neighbors" instead of "any neighbors", which reveals an advanced feature: Xling can determine whether the query point has more than τ neighbors, where τ is a user-determined number. And Xling downgrades to a general MSBF when $\tau = 0$. We call such a feature "filtering-by-counting".

By learning the data distribution, Xling solves the data-unawareness problem, and essentially as a regressor, it is not limited to any specific distance metric. Note that we have mentioned three thresholds until now, the *distance threshold* ϵ, the *Xling decision threshold* XDT, and the *neighbor threshold* τ. To make it clearer, we will indicate them respectively with "distance threshold", "decision threshold" and "neighbor threshold", or directly using their symbols.

Xling deploys novel optimization strategies to further improve the performance, including strategy to select ϵ values for effective training, and strategy to select the best XDT. Furthermore, Xling can be applied as a plugin onto many existing similarity join methods to significantly enhance their efficiency with tiny or no quality loss. We have applied Xling to a brute-force nested-loop similarity join, named **XJoin**, which achieves significantly higher efficiency (up to 14.6x faster) than the existing high-performance similarity join methods (some are used in industry), with a guarantee of high quality. In these applications, filtering-by-counting enables Xling to help the base similarity join method ignore the queries having only a trivial number of neighbors (e.g., 3 or 5, or even 50), which makes the acceleration more significant with tiny sacrifice of recall. In addition, we also apply Xling onto those prior methods and show that they are substantially accelerated with slightly more quality loss. Finally, XJoin and Xling are evaluated on their generalization capability, and the results prove that they can be transferred to updated or fully new dataset without re-training the learning model. To our best knowledge, We are the first to propose such a learning-based MSBF and XJoin is among the first practical filter-based similarity join methods for high-dimensional data.

The main contributions of this paper are as follows:

1) We propose XJoin, the first filter-based similarity join method for high-dimensional data, which is both efficient and effective.
2) We propose Xling, a generic framework for constructing learned metric space Bloom filters with general regression models.
3) We design performance optimization strategies in Xling, including selection of ϵ values for effective training and adaptive computing of XDT.
4) We conduct extensive evaluation to show the efficiency, effectiveness, usefulness and generalization of Xling and XJoin, as well as the remarkable performance improvement by applying Xling to other similarity join methods.

The rest of this paper is organized as follows: Section II introduces the prior works related to the techniques in this paper. Section III formally defines the key problems studied by this paper and the important notations being used. Section IV presents the architecture of Xling and the workflow of applying Xling to enhancing similarity join. Section V discusses details about the optimization strategies integrated in Xling. And Section VI reports and analyzes the evaluation results.

II. Related work

Learned Bloom filter: The learned Bloom filter is first proposed by [28] which treats the existence checking in Bloom filter as a classification task, thereby replaces the traditional Bloom filter with a deep learning based binary classifier followed by an overflow Bloom filter (to double check the negative outputs of the classifier to guarantee a zero false negative rate). And there have been many following works that further improve the learned Bloom filter [29], [30], [31] by adding auxiliary components or improving the performance of the hashing functions being used.

Learned cardinality estimation: Learning models have been widely used to predict the number of neighbors for a metric space range search, which is called "learned cardinality estimation". The learned cardinality estimation techniques

treat the task as a regression problem and solve it using regression models. The state-of-the-art methods [32], [33], [34], [35] are usually based on deep regression models to effectively learn the data distribution and make more accurate prediction than the non-deep approaches. Recursive Model Index(RMI) [36] is a hierarchical learning model that consists of multiple sub-models, where each sub-model is a regression model like neural network. CardNet [33] consists of a feature extraction model and a regression model. The raw data is transformed by the feature extraction model into Hamming space, which will then used by the regression model to predict its cardinality. SelNet [35] predicts the cardinality by a learned query-dependent piece-wise linear function.

Metric space Bloom filter: DSBF [26] and LSBF [21] are two of the representatives for metric space Bloom filter (MSBF), and following them many MSBF variants and relevant applications have been developed. [25] enables LSBF to handle multiple Euclidean distance granularity without rebuilding the data structure. [24] extends [25] to Hamming distance. [23] proposes a DSBF variant with zero false negative rate theoretically. Since most of those works are built on top of LSH, their effectiveness is usually limited by unknowing of the data distribution.

Similarity join: This problem can be further classified into two sub-types, exact and approximate similarity join, where the former is to exactly find all the truly close points while the latter allows some errors.

One family of the state-of-the-art efficient methods for exact similarity join is the epsilon-grid-order (EGO) based methods, including EGO-join [15], EGO-star-join [16], Super-EGO [17], FGF-Hilbert join [14], etc. They work by splitting the space into cells and sorting the data points along with those cells, which will then help reduce unnecessary computation in the similarity join.

Approximate similarity join methods are usually based on LSH [18], [37], [19], [4], [5], [12], [13]. The problem of those methods is data unawareness, i.e., the hashing-based space partitioning usually does not consider the data distribution, which may leads to imbalanced partitions that significantly lower the overall search performance [20].

III. Preliminaries

Notation	Description
ϵ	The distance threshold for range search and similarity join
τ	The neighbor threshold, i.e., whether or not a query has more than τ neighbors
XDT	The Xling decision threshold to classify the prediction as positive or negative
N	Dataset size
$\|R\|$, $\|S\|$	The sizes of set R and S
MAE, MSE	Mean absolute error and mean squared error
FPR, FNR	False positive rate and false negative rate

TABLE I: List of notations used in this and following sections

This section presents the critical notations frequently used in this paper and defines the key problems being studied, i.e., range search and similarity join. The notations are listed and explained in Table I, and more details of them are introduced where they are first referred in this paper.

In multi-dimensional space, range search is defined as the search of all data points whose distance to the query is smaller than a given threshold under some distance metric. And similarity join is to find all close point pairs whose distance is less than a threshold between two datasets. The formal definitions are as below:

Definition 1 (range search): Given a dataset $P = \{p_i | i = 1, 2, ..., n\}$, a distance metric $d(\cdot, \cdot)$, a query point q and a radius/threshold ϵ, range search tries to find a set of data points P^* such that for any $p_i^* \in P^*$, $d(q, p_i^*) \leq \epsilon$.

Definition 2 (similarity join): Given two datasets R and S, a distance threshold ϵ, and a distance metric $d(\cdot, \cdot)$, the similarity join between R and S is denoted by $R \bowtie_\epsilon S$, which combines each point $r \in R$ with each point $s \in S$ that is close/similar enough to r (i.e., with distance smaller than or equal to ϵ). Formally

$$R \bowtie_\epsilon S = \{(r, s) | \forall r \in R, \forall s \in S \ where \ d(r,\ s) \leq \epsilon\} \quad (1)$$

There are three critical "thresholds" in this paper, as listed in Table I, the "distance threshold" ϵ, the "neighbor threshold" τ and the "Xling decision threshold" XDT. To avoid confusion, we further explain them here:

1) The distance threshold ϵ is part of the range search and similarity join operations, which indicates the search range as shown in Definition 1 and 2.
2) τ is a threshold for the *groundtruth* neighbors. If a query truly has more than τ neighbors, we call it a *groundtruth positive*, otherwise it is a *groundtruth negative*.
3) The Xling decision threshold (XDT) is the threshold to classify the query based on the **predicted** neighbors. If a query is predicted as having more than XDT neighbors, we name it as a *predicted positive*, otherwise it is predicted negative.

Note that we do not directly use τ to threshold the predictions. This is because different cardinality estimation models have different prediction accuracy, leading to different predicted values for the same query. So it is necessary to use an adpative threshold driven by both model and data to classify the predictions, which is XDT, such that we can control the false positive or negative rate. More details are introduced in Section V-A.

IV. Architecture and workflow

A. The core and optimization strategies

Figure 1 illustrates the overall architecture and workflow of Xling. The core is the learned cardinality estimator and XDT (yellow boxes). The green boxes represent the optimization strategies deployed in the training and predicting stages. The blue shapes stand for the training data, including the raw data (which is input from the external) and the prepared data

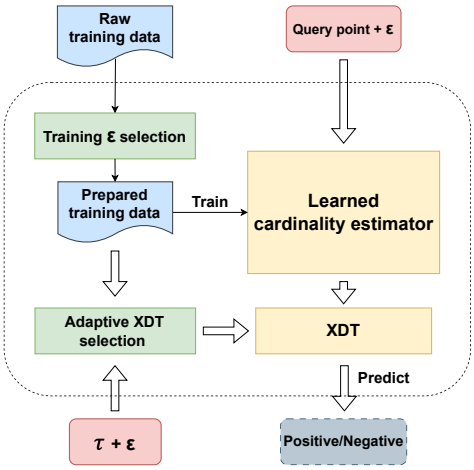


Fig. 1: Architecture and workflow of Xling

(which is generated based on the raw data inside Xling). The workflow of Xling mainly relies on the prepared training data instead of the raw data. In addition, the pink boxes indicate the query-time inputs, including the query point, ϵ and τ, where the query point and ϵ are fed into the learned cardinality estimator while the ϵ and τ are used to compute XDT. The grey box stands for the output, i.e., whether the query point is positive (with enough neighbors) or negative (having insufficient neighbors). For the workflow, the solid arrows present the offline training workflow, while the hollow arrows indicate the online workflow for querying and XDT computing.

The overall architecture includes (1) the core learned cardinality estimator and XDT and (2) the surrounding optimization strategy modules. [28] uses a classifier and a decision threshold to build a learned Bloom filter, while we use a regressor and the XDT to construct Xling, which is similar to the learned accelerator framework (LAF) in [38]. Specifically, LAF proves that learned cardinality estimator with a proper decision threshold can significantly accelerate range-search based high-dimensional clustering. As similarity join is also based on range search, such a solution works for it too. And unlike the case of learned Bloom filter, since filtering-by-counting requires estimating the specific number (mentioned in Section I), regressor is the best choice rather than classifier. But in LAF, the decision threshold has to be determined by grid search. To overcome this shortcoming, we design an *adaptive XDT selection* strategy based on the training data such that XDT can be efficiently computed. More details are introduced in Section V-B.

To reduce the false results, in addition to adaptively selecting a proper XDT, we also propose an adaptive *training* ϵ *selection* strategy to sample the most representative ϵ values for training the cardinality estimator effectively and thereby reducing the prediction error. The existing learned cardinality estimation studies (e.g., [39]) usually select the training ϵ values uniformly, which is not optimal, and we show in this paper that our proposed strategy generates a more representative training set and leads to a more effective model training. This training ϵ selection strategy is further discussed in Section V-A.

B. Training and querying workflows

There are two workflows in Xling, the offline training workflow and online querying workflow. In Figure 1, the solid arrows illustrate the training workflow. As introduced in Definition 2 and Section I, the two point sets to be joined are denoted by R and S, and without loss of generality, we assume that the size of R ($|R|$) is larger than size of S ($|S|$). Then R is used as the training set (i.e., the "raw training data" in Figure 1) to train Xling while S acts as the queries. The raw training data includes all points in R without ϵ information.

1) Training: The first step in the training workflow is concatenating the selected ϵ values onto each training point $r_i \in R$ (where $i = 1, 2, ..., |R|$), generating the "prepared training data" that is a Cartesian product between the point set R and ϵ set $\mathcal{E}$, i.e., the prepared training data is the set $\{(r_i, \epsilon_j) \mid \forall (r_i, \epsilon_j) \in R \times \mathcal{E}\}$, where r_i is a multi-dimensional vector and ϵ_j is a real number. Here the set $\mathcal{E}$ is generated by the training ϵ selection strategy (Section V-A).

To make it clearer, suppose R is a toy raw training set of 2 points r_a and r_b, and $\mathcal{E}$ includes two selected values ϵ_1 and ϵ_2. Then the prepared training set looks like $\{(r_a, \epsilon_1), (r_a, \epsilon_2), (r_b, \epsilon_1), (r_b, \epsilon_2)\}$ associated with their targets (i.e., the groundtruth numbers of neighbors). Then the learned cardinality estimator is trained with the prepared training data.

2) Querying: The querying workflow is presented with hollow arrows in Figure 1. First, ϵ and τ are input to compute the XDT using the adaptive XDT selection strategy based on the prepared training data. Second, the query point and the ϵ are concatenated and fed to the cardinality estimator to predict the number of neighbors. Finally, the prediction is compared with XDT and the answer (positive or negative) is determined.

C. Use in similarity join

Use of Xling to speed up similarity join is straightforward: given ϵ, τ, R and S, supposing $|R| > |S|$, Xling is first trained on R, then for each query point $s \in S$, Xling predicts whether s has enough neighbors in R under the ϵ. If yes, the range search (either a brute-force search or an indexed search like using LSH) will be executed in R for query s, otherwise s is predicted as negative and the search for it will be skipped. In this way, unnecessary range search is reduced and the join efficiency is improved.

As a generic filter, Xling can be applied onto any nested-loop based similarity join algorithms to speed them up. When using deep regressor as the core cardinality estimator in Xling, its prediction time complexity for each query is constant (O(1)) with the data scale, which in practice can be further accelerated by GPU. And the training time is not an issue due to the generalization capability of the deep models, i.e., a trained estimator can be used on any other dataset with similar distribution, which is proved by our evaluation in Section VI-F.

We implement XJoin by applying Xling onto a naive nested-loop similarity join method which does a brute-force range

search for each query. Our evaluation presents that XJoin outperforms the state-of-the-art similarity join baselines. We also apply Xling to some approximate similarity join methods, which shows Xling successfully improves their speed-quality trade-off capability.

V. Optimization strategies

In this section we introduce the details for the optimization strategies we propose in Xling.

A. Training ϵ selection

[39] uniformly samples the training ϵ from range [0, θ_{max}], where θ_{max} is a user-determined upper limit for the distances. However, such a selection strategy is not optimal. We show in this section that the cardinality estimation models can be trained more effectively by using our data-aware threshold selection strategy. Furthermore, our strategy can be generalized to better solve a family of regression problems.

The specific family of regression problems is named *Condition-based Regression* (shortly denoted by *CR*) and defined as follows:

Definition 3 (Condition-based Regression problem): Given a dataset $\mathcal{D}$, a condition c associated with each data point $p \in \mathcal{D}$, and a target t corresponding to each (p, c) pairs, the CR problem is to learn the relationship between the presented (p, c) and t, then predict the corresponding t for any given (p, c).

In the context of learned cardinality estimation, the condition is the ϵ while the target is the number of neighbors.

In most Condition-based Regression problems, the condition is usually a continuous variable (like distance) whose values cannot be fully enumerated, therefore it is worth studying how to sample the conditions for better training the regression model in order to lower prediction error, which is formally defined as such:

Definition 4 (Training condition selection for CR): Given a dataset $\mathcal{D}$ of size n, a set of m candidate values $\{c_{i1}, c_{i2}, ..., c_{im}\}$ of the condition c_i for each data point $p_i \in \mathcal{D}, 1 \leq i \leq n$, the corresponding target t_{ij} for each $(p_i, c_{ij}), 1 \leq j \leq m$, and a sampling number s, the training condition selection task is to select s pairs from the m (c_{ij}, t_{ij}) pairs to form s training tuples for each point p_i, which results in totally sn training tuples for the whole dataset $\mathcal{D}$, such that the mean regression error is minimized.

Note that our discussion starts from a discrete candidate set, instead of directly beginning from the continuous range (like [0, θ_{max}]). This is just to unify the discussion between continuous-range uniform sampling and our sampling method which requires preprocessing to discretize the range, which will not lose generality. m is usually large (such that the values are dense enough) to approximate the case of the continuous condition values, so we do not use all the m candidate values to generate mn training tuples, as the memory space is limited. For example, in our evaluation, $m = 100$, in which case mn tuples require more memory space than that of our evaluation machine. Therefore only the s sampled conditions are used to form the training tuples.

Uniformly sampling training conditions cannot reflect the unevenly data distribution in real-world datasets. Therefore, we design a generic adaptive strategy to further fine-tune the initial uniformly sampling conditions for CR problem based on the density of the targets, such that the resulting training tuples are more representative for the data distribution of the whole dataset. Our adaptive training condition selection (ATCS) strategy (Algorithm 1) follows the steps below for each data point p in the training set:

1) Given the uniformly sampled candidate conditions $\mathcal{C}_p$ and the corresponding targets $\mathcal{T}_p$ for point p (i.e., when condition $c_i \in \mathcal{C}_p$ is applied to point p, the target is correspondingly $t_i \in \mathcal{T}_p$), the minimum and maximum targets are found (t_{min} and t_{max}, line 5) and the interval $[t_{min}, t_{max}]$ is then partitioned into s bins evenly (line 6).
2) Then each candidate condition is mapped into one bin based on its paired target (line 7-8).
3) Finally the specific number of condition-target pairs are sampled from each bin (line 10-11) according to the fraction of the bin size ($|\mathcal{B}_i|$) over total number of the conditions ($|\mathcal{C}_p|$), where bin size is the number of pairs in that specific bin.
4) Since some bins may generate zero samples (i.e., when $s|\mathcal{B}_i| < |\mathcal{C}_p|$), the final sampled condition-target pairs may be not enough given the required s. In such a case, the rest[1] of the pairs will be randomly chosen from the unselected ones that are not yet included in the samples above (line 12-13).

The final training set for the whole $\mathcal{D}$ is the collection of the training tuples (p, c, t) generated by the strategy for each $p \in \mathcal{D}$, i.e., the final training set includes $s|\mathcal{D}|$ tuples.

Unlike uniformly sampling, ATCS is data-aware by binning the condition-target pairs to estimate the density of the targets and then sampling the final conditions based on the density, which generates more representative training conditions and targets. For example, let $m = 100, s = 5$, i.e., the interval of targets [t_{min}, t_{max}] is split into 5 bins evenly and their corresponding condition values are placed into the bins accordingly. Supposing the numbers of conditions from $\mathcal{B}_1$ to $\mathcal{B}_5$ are 59, 1, 19, 1, 20, the uniformly sampling will select 2, 1, 0, 1, 1 conditions from them (i.e., selecting one per 20), while ATCS will first select 2, 0, 0, 0, 1 from them (Algorithm 1 Line 11) then probably select the rest 2 conditions from $\mathcal{B}_1$, $\mathcal{B}_3$ and $\mathcal{B}_5$ which are the most dense areas (Algorithm 1 Line 13). In this example, the uniformly sampling gets 2 out of the 5 sampled conditions from very sparse areas ($\mathcal{B}_2$ and $\mathcal{B}_4$) in the distribution of targets, which cannot well reflect the overall distribution, while ATCS probably selects all the 5 from dense areas and results in more representative training

[1] In our evaluation, the rest of the pairs (which are randomly chosen) usually occupy 10% $\sim$ 20% in the final training set for the whole $\mathcal{D}$, i.e., $0.1s|\mathcal{D}| \sim 0.2s|\mathcal{D}|$

Algorithm 1 Adaptive training condition selection (ATCS) strategy

Input: Dataset $\mathcal{D}$, map from each data point to its uniformly sampled candidate condition list $\mathcal{C}$, map from each point to its target list $\mathcal{T}$, sampling number s
Output: set of resulting training tuples $\mathcal{R}$

$\mathcal{R} := \emptyset$
for each point p in $\mathcal{D}$ **do**
  $\mathcal{C}_p := \mathcal{C}(p)$ ▷ the condition list for p
  $\mathcal{T}_p := \mathcal{T}(p)$ ▷ the target list for p, one-to-one corresponding to $\mathcal{C}_p$
  $t_{min}, t_{max} :=$ min($\mathcal{T}_p$), max($\mathcal{T}_p$)
  Split interval $[t_{min}, t_{max}]$ into s bins evenly
  for each c in $\mathcal{C}_p$ and corresponding t in $\mathcal{T}_p$ **do**
    Place (c, t) into a bin bounded by $[t_a, t_b)$ such that $t \in [t_a, t_b)$
  $\mathcal{S} := \emptyset$ ▷ the selected (c, t) collection for p
  for each bin $\mathcal{B}_i$ **do**
    $\mathcal{S} := \mathcal{S} \cup \{ \left\lfloor \frac{s|\mathcal{B}_i|}{|\mathcal{C}_p|} \right\rfloor$ randomly sampled (c, t) pairs from $\mathcal{B}_i\}$
  if $|\mathcal{S}| < s$ **then**
    Fill with random samples from unselected (c, t) until $|\mathcal{S}| = s$
  $\mathcal{R} := \mathcal{R} \cup \{(p, c_s, t_s) | (c_s, t_s) \in \mathcal{S}\}$ ▷ combine all pairs in $\mathcal{S}$ with p
return $\mathcal{R}$

samples, by which the model can be trained more effectively. Our evaluation in Section VI-B shows that ATCS helps reduce 50% $\sim$ 98% of the prediction error (MAE and MSE).

B. Xling decision threshold (XDT) selection

Xling decision threshold (XDT) determines whether the prediction means positive. Its value is influenced by τ, the way it is computed, and the way to identify the groundtruth negative training samples, which will be introduced in this section. Note that XDT is determined purely based on the training set offline, regardless of the online queries, i.e., the computation of XDT does not raise overhead on the online querying.

Basically, XDT is computed using the groundtruth negative training samples (i.e., the training points with no more than τ neighbors). We propose two ways to compute XDT: false positive rate (FPR) based and mean based.

1) FPR-based XDT selection: similarly to [36], [40], [30], given a FPR tolerance value t_{fpr} (e.g., 5%), this method lets the estimator make predictions for the training points and sets XDT such that the resulting filter FPR on training set is lower than t_{fpr}.
2) mean-based XDT selection: this method sets XDT to be the mean predicted value for all the groundtruth negative training samples.

In our evaluation (Section VI-B2), FPR-based method usually results in a higher XDT, leading to more speedup and lower quality in end-to-end similarity join, so we design the mean-based method to provide the second option which results in a lower XDT and leads to higher quality and less speedup. They are useful in different situations as shown in our evaluation. In addition, no matter using FPR-based or mean-based selection, there is a trend that a larger τ will result in a higher XDT, which is straightforward to understand: a larger τ causes training samples with more neighbors to be negative, therefore cardinalities of the negative samples increase overall, making the computed XDT increased.

But both methods have a problem: we have to first identify the groundtruth negative samples in the training set, which is costly in the high-dimensional cases. Since the negative samples depend on ϵ, and the training set only include a tiny portion of all possible ϵ, the queried ϵ will probably not exist in training set (named "out-of-domain ϵ"), in which case intensive range search has to be executed to compute the negative samples from scratch.

Therefore, it is non-trivial to study how to easily get the training targets (i.e., groundtruth cardinalities) under the out-of-domain ϵ such that the groundtruth negative samples can be identified without doing range search for each training point. We propose an interpolation-based strategy to generate the approximate targets. Specifically, given a point, its ϵ-cardinality curve is monotonically non-decreasing, i.e., with ϵ increasing, the cardinality will never decrease. So we can approximate the curve segment between two neighboring training ϵ values (denoted by ϵ_1 and ϵ_2) as linear, and use linear-interpolation to estimate the groundtruth cardinality for any training point under a out-of-domain ϵ_3 between ϵ_1 and ϵ_2, as shown in Equation 2.

$$t_3 = t_1 + \frac{t_2 - t_1}{\epsilon_2 - \epsilon_1}(\epsilon_3 - \epsilon_1) \tag{2}$$

where t_i is the target of the current training point under ϵ_i ($i = 1, 2$), t_3 is the approximate target under out-of-domain ϵ_3, and $\epsilon_1 < \epsilon_2$. The approximate targets are then used to find all groundtruth negative training samples under ϵ_3 based on which XDT is computed.

Our evaluation shows that in most cases, Xling deploying the interpolation-based method has a competitive prediction quality to that using the naive solution. Furthermore, there are two advantages of the proposed method over the naive way: (1) it is significantly faster since no range search is executed, and (2) as in Section VI-B2, interpolation-based method tends to result in no higher false negative rate (FNR) than the naive method, which is better for the effectiveness (e.g., the recall) of Xling.

VI. Experiments

A. Experiment settings

Environment: All the experiments are executed on a Lambda Quad workstation with 28 3.30GHz Intel Core i9-9940X CPUs, 4 RTX 2080 Ti GPUs and 128 GB RAM.

Datasets: Table II provides an overview for our evaluation datasets, reporting their sizes, data dimensions and data types. We introduce more details here:

1) FastText: 1M word embeddings (300-dimensional) generated by fastText model pre-trained on Wikipedia 2017, UMBC webbase corpus and statmt.org news dataset.

Dataset	#Points	#Sampled	Dim	Type
FastText	1M	150k	300	Text
Glove	1.2M	150k	200	Text
Word2vec	3M	150k	300	Text
Gist	1M	150k	960	Image
Sift	1M	150k	128	Image
NUS-WIDE	270k	150k	500	Image

TABLE II: Evaluation dataset information, including the number of total points in the whole dataset (*#Points*), the number of sampled points for evaluation (*#Sampled*), data dimension (*Dim*), and the raw data type (*Type*).

2) Glove: 1.2M word vectors (200-dimensional) pre-trained on tweets.
3) Word2vec: 3M word embeddings (300-dimensional) pre-trained on Google News dataset.
4) Gist: 1M GIST image descriptors (960-dimensional).
5) Sift: 1M SIFT image descriptors (128-dimensional).
6) NUS-WIDE: 270k bag-of-visual-words vectors (500-dimensional) learned on a web image dataset created by NUS's Lab for Media Search.

Due to computing resource limitation, we randomly sample 150k vectors from each of them for the evaluation. In the rest of this paper, any mentioned dataset name is by default meaning the 150k subset of the corresponding dataset.

We then normalize the sampled vectors to unit length because (1) this makes the distances bounded, i.e., both cosine and Euclidean distance are within [0, 2] on unit vectors, making it easier to determine ϵ, and (2) some baseline methods do not support cosine distance, in which case we have to convert the cosine ϵ into equivalent Euclidean ϵ for them on unit vectors, as in our previous work [38]. We then split each dataset into training and testing sets by a ratio of 8:2, where the training set acts as R while the testing set is S. Xling is trained on the training set, then all the methods are evaluated using the corresponding testing set as queries.

Learning models: we use several learning models to evaluate the performance of the optimization strategies in Xling. They are introduced as follows. For the deep models, we use the recommended configurations in prior works, while for non-deep models a grid search is executed to find the best parameters. Specifically, (1) RMI [28]: We use the same configuration in [38], i.e., three stages, respectively including 1, 2, 4 fully-connected neural networks. Each neural network has 4 hidden layers with width 512, 512, 256, and 128. The RMI is trained for 200 epochs with batch size 512. (2) NN: We also evaluate the neural network (NN), which is simply a single sub-model extracted from the RMI above. So all the parameters (including the training configuration) are same as that in RMI. (3) SelNet [41]: We use the the same model configuration as in the paper. (4) XGBoost Regressor (XGB), LightGBM Regressor (LGBM) and Support Vector Regressor (SVR): These are all non-deep regressors, we do a grid search to determine their best parameters.

Similarity join baselines: The evaluation baselines include both exact and approximate methods, where the latter are the major baselines.

1) Naive: This is a brute-force based nested-loop similarity join, i.e., for each query point in S, do a brute-force range search for it in R. Its results act as the groundtruth for measuring the result quality of all other methods.
2) SuperEGO[2] [17]: This is an exact method based on Epsilon Grid Ordering (EGO) that sorts the data by a specific order to facilitate the join.
3) LSH: This is an approximate method using LSH. First the points in R are mapped into buckets by LSH, then each query is mapped to some buckets by the same LSH. The points in those buckets and nearby buckets will then be retrieved as candidates and verified. This method is implemented using FALCONN [42], the state-of-the-art LSH library.
4) KmeansTree: This is an approximate method using K-means tree, where the tree is built on R and the space is partitioned and represented by sub-trees. Then each query is passed into the tree and a specific sub-tree (which represents a sub-space of R) will be inspected to find neighbors within the range. We use FLANN [43], a widely used high-performance library for tree-based search, to implement this method.
5) Naive-LSBF: This is an approximate filter-based method that simply applies LSBF [3] onto the Naive method, in the same way as the use of Xling in similarity join (Section IV-C). We use LSBF instead of the following MSBF variants because those variants raise unnecessary overhead to support specific extra features.
6) IVFPQ: we also select an approximate nearest neighbor (ANN) index as part of the baselines, which is the IVFPQ index [44] in FAISS [45], one of the industrial ANN search libraries. Since IVFPQ does not support range search natively, we evaluate it by first searching for a larger number of nearest candidates, then verifying which candidates are the true neighbors given ϵ. And as IVFPQ tends to achieve a high search speed with relatively lower quality (as discussed in our previous work [46]), this baseline does not make much sense in the fixed-parameter end-to-end evaluation (Section VI-D). Therefore, we only evaluate it in the trade-off (Section VI-E) and generalization (Section VI-F) experiments.

Our proposed methods: Following the way described in Section IV-C, we apply Xling to several base similarity join methods mentioned above. The resulting methods are named as: (1) XJoin (2) LSH-Xling (3) KmeansTree-Xling (4) IVFPQ-Xling. The *XJoin* is our major proposed method that is evaluated in all the similarity join experiments, while the other proposed methods here are only compared with the corresponding baseline methods to show the enhancement brought by Xling to them. The learned cardinality estimator used by Xling in all these methods is an RMI. Note that

[2]code available at https://www.ics.uci.edu/~dvk/code/SuperEGO.html
[3]code available at https://github.com/csunyy/LSBF

the goal of this paper is to reveal the potential of such a new framework on speeding up similarity join generically, so selecting the best estimation model is out of scope. Given that RMI has been used as a strong baseline for learned cardinality estimation in [39] and it is not the most state-of-the-art, it is a fair choice for Xling in the evaluation, especially when we can show that RMI is already good enough to outperform the other baselines. We will evaluate Xling with other estimators in the future work.

In XJoin, the XDT is computed by FPR-based selection (introduced in Section V-B) with 5% FPR tolerance, and $\tau = 50$, while in LSH-Xling, KmeansTree-Xling and IVFPQ-Xling, XDT is computed by mean-based selection with $\tau = 0$. As discussed in Section V-B, "mean-based XDT selection + smaller τ" leads to higher quality while "FPR-based selection + larger τ" results in more acceleration. Since LSH, KmeansTree and IVFPQ are approximate methods that sacrifice quality for efficiency, mean-based XDT with lowest τ can accelerate them while minimizing the further quality loss. For Naive, the bottleneck is the efficiency, so we choose the other configuration for Xling in order to achieve a non-trivial speedup.

Metrics: The distance metric for text data is cosine distance while that for image data is Euclidean distance. For the baselines which do not support cosine distance, we follow [38] to equivalently convert cosine distance to Euclidean distance. The evaluation metrics include (1) end-to-end join time for measuring the efficiency of similarity join methods, (2) recall (i.e., the ratio of the returned positive results over all the groundtruth positive results) for measuring the similarity join result quality, (3) mean absolute error (MAE) and mean squared error (MSE) for measuring prediction quality of the learned cardinality estimator, and (4) false positive rate (FPR) and false negative rate (FNR) for measuring prediction quality of Xling.

Evaluation ϵ: As we have normalized all the vectors, the distances between them are bounded, i.e., [0, 2] for both cosine and Euclidean distances. Since many use cases of similarity join usually choose ϵ from the range [0.2, 0.5] [9], [10], [3], [47], we do a grid search in this range and determine the representative evaluation ϵ values: 0.4, 0.45 and 0.5, based on the portion of negative queries (i.e., the queries having no neighbor in R) in S. We set the upper limit for the portion as 90% as too many negative queries will make the evaluation unconvincing. Under the selected ϵ values, most of the datasets (except NUS-WIDE) have a proper portion that is no more than 90%. Table III reports the portion of negative queries for each dataset under each ϵ.

B. Optimization strategy evaluation

In this section we report and analyze the evaluation results of the optimization strategies.

1) Training ϵ selection: The results of the training ϵ selection strategy is reported in Table IV. Due to the space limit, we only present the results for 4 out of 6 datasets.

Dataset	Portion ($\epsilon = 0.4$)	Portion ($\epsilon = 0.45$)	Portion ($\epsilon = 0.5$)
FastText	0.1103	0.0443	0.0116
Glove	0.8668	0.7851	0.6637
Word2vec	0.2875	0.1675	0.0803
Gist	0.8442	0.3939	0.1027
Sift	0.5578	0.3494	0.1531
NUS-WIDE	0.9743	0.9653	0.9544

TABLE III: The portion of negative queries for each dataset under each ϵ

Following Definition 4, to make it simple, we have the set of candidate condition values shared by all the training points, i.e., the set $\{c_{i1}, c_{i2}, ..., c_{im}\}$ is same for any training point p_i. We let $m = 100$ and construct such a set by evenly sampling 100 values from a range $[c_{min}, c_{max}]$, where we set $c_{min} = 0.4, c_{max} = 0.9$ for cosine distance while $c_{min} = 0.5, c_{max} = 2.0$ for Euclidean distance. These c_{min} and c_{max} values are selected based on a grid search given the evaluation experience from our previous work [38]. Then we set the sampling number $s = 6$, i.e., for each training point, 6 distinct condition values will be sampled from the 100 candidate values and become the final training ϵ values to be paired with the point in the prepared training data. Two sampling strategies are deployed to select the 6 values: (1) uniform sampling, e.g., selecting the 1st, 20th, 40th, 60th, 80th and 100th values (2) our ATCS strategy (Algorithm1). The former is presented as "fixed" while the latter is marked as "auto" in the *Strategy* columns of Table IV.

The regression models (i.e., the learning models listed in Section VI-A) will first be trained respectively using the two prepared training sets from different strategies, then they will do inference on some prepared testing sets and the inference quality is measured by MAE and MSE, which can reflect the training effectiveness. For a fair testing, we generate two kinds of prepared testing sets: (1) the testing points with randomly selected ϵ values from the 100 candidates mentioned above, and (2) the same testing points with uniformly selected ϵ values from the candidates. The inference quality on the former set is reported in the *Random Testing* ϵ columns while that on the latter is reported in the *Uniform Testing* ϵ columns of Table IV. And for each learning model, the *Strategy* which results in better inference quality (i.e., lower MAE and MSE) is highlighted by bold text.

In most cases, the "auto" strategy (i.e., our ATCS strategy) achieves a better inference quality (e.g., reducing up to 98% of the MSE on NUS-WIDE dataset) than uniform training ϵ, meaning that ATCS strategy is generic to facilitate various kinds of regression models and highly effective to raise a significant improvement on the prediction quality. Therefore, for all the following experiments in this paper, the training sets are prepared by ATCS.

2) XDT selection: As mentioned in Section V-B, XDT is influenced by three factors: (1) τ, (2) the XDT selection method (mean-based or FPR-based), and (3) the way to get training targets for out-of-domain ϵ (interpolation-based

Random Testing ϵ		Uniform Testing ϵ		Model	Strategy
MAE	MSE	MAE	MSE		
4.04	6.69	2.57	2.68	XGB	fixed
2.54	**1.32**	**2.54**	**1.26**	XGB	**auto**
3.66	6.65	2.14	2.57	LGBM	fixed
1.8	**0.83**	**1.79**	**0.81**	LGBM	**auto**
23.79	97.16	27.55	127.15	SVR	fixed
22.26	**81.06**	**25.34**	**106.22**	SVR	**auto**
96.13	24711.7	96.44	28299.85	SelNet	fixed
93.8	**22228.79**	**94.39**	**25737.67**	SelNet	**auto**
2.08	1.39	1.69	1.00	NN	fixed
0.39	**0.03**	**0.44**	**0.05**	NN	**auto**
2.55	5.72	1.52	2.18	RMI	fixed
0.20	**0.01**	**0.19**	**0.01**	RMI	**auto**

(a) Sift

Random Testing ϵ		Uniform Testing ϵ		Model	Strategy
MAE	MSE	MAE	MSE		
5.32	**22.84**	**7.81**	**48.16**	XGB	**fixed**
5.02	24.09	7.89	50.7	XGB	auto
5.29	20.46	7.45	43.82	LGBM	fixed
4.49	**19.61**	**7.1**	**43.52**	LGBM	**auto**
9.01	**46.41**	12.6	**84.58**	SVR	fixed
8.7	50.18	**12.32**	90.46	SVR	**auto**
8.91	**54.69**	**12.57**	**97.23**	SelNet	**fixed**
8.91	54.7	12.57	97.24	SelNet	auto
4.38	18.87	6.78	41.95	NN	fixed
3.33	**11.85**	**5.09**	**23.19**	NN	**auto**
4.52	19.44	6.85	42.57	RMI	fixed
3.43	**13.84**	**5.35**	**27.51**	RMI	**auto**

(b) Word2Vec

Random Testing ϵ		Uniform Testing ϵ		Model	Strategy
MAE	MSE	MAE	MSE		
4.69	5.49	5.02	5.36	XGB	fixed
2.53	**1.43**	**2.55**	**1.49**	XGB	**auto**
4.7	5.49	4.76	4.94	LGBM	fixed
2.45	**1.28**	**2.47**	**1.33**	LGBM	**auto**
27.61	141.55	29.89	169.73	SVR	fixed
23.45	**81.06**	**24.5**	**94.06**	SVR	**auto**
36.19	357.57	38.14	387.45	SelNet	fixed
36.19	**357.52**	**38.14**	**387.39**	SelNet	**auto**
0.54	0.14	0.79	0.36	NN	fixed
0.32	**0.03**	**0.38**	**0.05**	NN	**auto**
2.09	2.23	2.80	4.49	RMI	fixed
0.67	**0.18**	**0.77**	**0.24**	RMI	**auto**

(c) FastText

Random Testing ϵ		Uniform Testing ϵ		Model	Strategy
MAE	MSE	MAE	MSE		
4.74	14.99	3.4	6.35	XGB	fixed
3.36	**2.62**	**3.37**	**2.69**	XGB	**auto**
4.23	15.29	2.89	6.4	LGBM	fixed
2.4	**2.6**	**2.36**	**2.46**	LGBM	**auto**
22.81	**70.71**	**24.6**	**80.33**	SVR	**fixed**
25.22	99.4	25.06	97.04	SVR	auto
82.26	8655.88	82.62	9886.03	SelNet	fixed
75.96	**6560.3**	**75.93**	**7490.16**	SelNet	**auto**
3.39	10.56	2.51	5.01	NN	fixed
0.83	**0.20**	**1.04**	**0.33**	NN	**auto**
4.24	19.66	2.49	6.60	RMI	fixed
0.48	**0.27**	**0.39**	**0.13**	RMI	**auto**

(d) NUS-WIDE

TABLE IV: Effectiveness of the two training ϵ selection strategies for different regressors trained on different datasets, where *MAE* columns show the original numbers multiplied by 10^{-3} and *MSE* columns show the original numbers multiplied by 10^{-7}

Dataset	Model	ϵ	XDT Selection	Approximate Targets				Exact Targets			
				FPR	FNR	Time(s)	XDT	FPR	FNR	Time(s)	XDT
Glove	XGB	0.4	mean	0.4948	0.3982	1.3618	-8.45	0.4948	0.3982	1037.9202	-8.45
			FPR	0.0562	0.8578	1.3859	406.83	0.0562	0.8578	1037.9278	406.83
		0.45	mean	0.5019	0.4247	2.5836	-10.66	0.4996	0.4259	1038.6554	-9.28
			FPR	0.0608	0.8737	2.5499	392.43	0.0548	0.882	1038.6161	405.70
		0.5	mean	0.4989	0.4373	2.51	-12.56	0.5004	0.4358	1038.3489	-13.50
			FPR	0.0602	0.8903	2.8515	387.51	0.0566	0.8946	1038.379	396.47
	RMI	0.4	mean	0.4071	0.2506	10.0679	4.38	0.4071	0.2506	1046.1885	4.38
			FPR	0.0501	0.5049	9.9112	6.46	0.0501	0.5049	1046.1892	6.46
		0.45	mean	0.4496	0.2308	10.9286	4.57	0.3537	0.2702	1046.9525	4.76
			FPR	0.0808	0.4902	11.1269	6.48	0.0506	0.5385	1047.0098	7.37
		0.5	mean	0.3613	0.2546	10.8683	5.15	0.3043	0.284	1046.6399	5.34
			FPR	0.0709	0.5041	11.1455	7.99	0.0518	0.5424	1046.5916	8.82
NUS-WIDE	XGB	0.4	mean	0.5041	0.2202	1.4501	-3.31	0.5011	0.2215	2638.2846	15.41
			FPR	0.0555	0.6554	1.5899	3424.20	0.0524	0.6645	2638.2916	3499.68
		0.45	mean	0.5024	0.2428	1.4116	-3.31	0.5008	0.2428	2655.1167	5.55
			FPR	0.0535	0.6727	1.4341	3424.20	0.0519	0.6756	2655.1188	3463.20
		0.5	mean	0.5009	0.2719	1.8672	-3.31	0.5009	0.2719	2651.2069	-3.31
			FPR	0.0516	0.6981	2.0873	3424.20	0.0516	0.6981	2651.1754	3424.20
	RMI	0.4	mean	0.3672	0.057	10.6236	2.86	0.3537	0.0596	2647.5025	2.99
			FPR	0.0534	0.0894	11.0576	9.87	0.048	0.092	2647.5929	10.45
		0.45	mean	0.3301	0.071	10.4377	2.43	0.3216	0.071	2664.379	2.52
			FPR	0.0504	0.1123	10.8119	9.55	0.0463	0.1161	2664.2925	9.95
		0.5	mean	0.2872	0.087	11.3239	2.18	0.2872	0.087	2660.6731	2.18
			FPR	0.0467	0.1294	11.3493	9.54	0.0467	0.1294	2660.3748	9.54

TABLE V: Prediction quality (FPR and FNR) of Xling when XDT is computed in different ways (while fixing $\tau = 0$). There are two dimensions for a way to compute XDT: (1) using mean-based or FPR-based XDT selection method (2) using interpolation-based method to approximate the targets or naive method to compute the exact targets. The time to compute the targets is also presented (*Time*). Results on other datasets are similar to Glove and NUS-WIDE. Note that we allow XDT to be less than zero.

approximate targets or naively computed exact targets). In this section we evaluate the last two factors due to page limit. Specifically, here we fix $\tau = 0$. For each dataset, the learned cardinality estimator of Xling is first trained using the training set, then we vary the setting for the two factors, compute XDT under each setting, and measure the FPR and FNR of Xling on the testing set of the current dataset, as well as the time for computing the training targets.

Due to space limit, Table V reports the results for two representative models (i.e., XGB for non-deep and RMI for deep model) on two datasets Glove (text) and NUS-WIDE (image). For the factor of target computing, the results show that the two target computing methods usually lead to similar FPR and FNR, while our proposed interpolation-based target approximation is around 100x $\sim$ 2000x faster than the naive target computing, meaning that interpolation-based target approximation is both effective and highly efficient. For the factor of XDT selection, as mentioned in Section V-B, the results present that FPR-based XDT selection usually generates a higher XDT than mean-based, thereby the former tends to determine more queries as negative and causes lower FPR and higher FNR than the latter. So in the end-to-end similarity join, using FPR-based XDT selection will lead to higher speedup but lower quality than mean-based, which supports the statements in Section V-B.

C. Filter effectiveness evaluation

In this section we evaluate the prediction quality between Xling and LSBF, reported by FPR and FNR of their predictions for the testing sets. We fix $\tau = 0$ as LSBF does not support the cases where $\tau > 0$.

The results are included in Table VI. In addition to FPR and FNR, the table also includes the total number of true neighbors found for all the testing queries (*#Nbrs*) by Naive-LSBF and XJoin, the number of predicted positive queries (*#PPQ*) which are the query points predicted as positive by the filter, and the average number of neighbors per predicted positive query (*#ANPQ*), i.e., #ANPQ = #Nbrs/#PPQ. We have the following observations: (1) In most cases, Xling with mean-based XDT has both lower FPR and FNR than LSBF, meaning it is more effective than LSBF. (2) In many cases, Xling with FPR-based XDT performs similarly to the observation above, while in some other cases, it has higher FNR than LSBF. However, even with a higher FNR, it still finds more true neighbors in those cases than LSBF, e.g., the case of Word2vec under $\epsilon = 0.5$. The reason is that the queries predicted as positive by Xling overall have more neighbors than those by LSBF (reflected by #ANPQ), due to which Xling can find more true results with less range search (i.e., the #PPQ of FPR-based Xling is usually less than LSBF). In short, Xling learns the data distribution and makes predictions based on the data density, such that it can maximize the recall with minimized number of search, which is a huge advantage over the data-unaware LSBF.

D. End-to-end evaluation

In end-to-end evaluation, the key parameters for the methods are fixed as such: (1) Naive and SuperEGO have no user-specific parameters. (2) In LSH, the number of hash tables $l = 10$, number of has functions $k = 18$, the number of hash buckets to be inspected in each table $n_p = 40$ for text datasets while $n_p = 2$ for image datasets. (3) In KmeansTree, the branching factor is fixed to be 3, the portion of leaves to be inspected $\rho = 0.02$ for text datasets while $\rho = 0.012$ for image datasets. (4) For Naive-LSBF, its LSH-related parameters k and l are the same as the method LSH, length of the bit array in LSBF is fixed to be 2,160,000 (i.e., $|R| \times k$), the parameter W in the p-stable LSH functions is set to be 2.5 for text datasets while $W = 2$ for image datasets. (5) For IVFPQ, the parameters for building the index are $C = 300$, $m = 32$ or $m = 25$ when the data dimension is not an integer multiple of 32, $b = 8$, $p = 50$, where C is the number of clusters, m is the number of segments into which each data vector will be split, b is the number of bits used to encode the clsuter centroids, and p is the number of nearest clusters to be inspected during search. In our implementation, this baseline first uses the IVFPQ index to select 1000 nearest neighbors then verifies them given ϵ. The number 1000 is determined since for all datasets, at least 50% testing queries have less than 1000 neighbors within ϵ, and in most datasets 1000 candidates are enough to find all correct neighbors for 70% $\sim$ 90% testing queries. (6) The configuration of Xling is introduced in Section VI-A, while the base methods in SuperEGO-Xling, LSH-Xling and KmeansTree-Xling use the same parameters as above.

The results are illustrated in Figure 2. SuperEGO has unknown bugs that prevent it from running on FastText and NUS-WIDE datasets, so Figure 2 does not include SuperEGO for these two datasets. SuperEGO relies on multi-threading for acceleration, so on some datasets its running time is even longer than the Naive method since we only use one thread to evaluate all the methods. The results show that our method XJoin has a higher speed (presented by the red bar) than all the baseline methods, as well as higher quality (reported by the red line) than the other approximate methods (i.e., LSH, KmeansTree and Naive-LSBF) in most cases. The recall of Naive and SuperEGO is always 1 as they are exact methods. Specifically, XJoin achieves up to 17x and 14.6x speedup respectively compared to the exact methods and approximate methods while guaranteeing a high quality. The results prove the high efficiency and effectiveness of our proposed method.

E. Speed-quality trade-off evaluation

In this evaluation, We fix the dataset and ϵ, then vary some key parameters of the approximate baselines and XJoin, i.e., n_p in LSH, the branching factor and the ratio ρ in KmeansTree, W and bit array length in Naive-LSBF, C and p in IVFPQ, and the XDT selection mode (mean or FPR based) and τ in XJoin. For LSH and KmeansTree, we always set the parameters of their Xling-enhanced versions the same as the original version, in order to make the comparison between them fair. The varied parameters result in varied end-to-end similarity join time and quality, based on which we get the speed-quality trade-off curves, as illustrated in Figure 3.

We select three cases to present in Figure 3: the datasets Glove, Word2vec and Gist with $\epsilon = 0.45$, and other cases are similar to them. According to the results, we conclude that (1) XJoin has better trade-off capability than the original version of the approximate baselines, i.e., LSH, IVFPQ, KmeansTree and Naive-LSBF, meaning that XJoin can achieve high quality with minimum processing time, or sacrifice tiny quality for significant efficiency gain. (2) The Xling-enhanced versions of the approximate baselines (i.e., LSH-Xling, IVFPQ-Xling and KmeansTree-Xling) have significant better trade-off capability than the original versions, which also means Xling successfully accelerates the original versions with a relatively negligible quality loss. This demonstrates the generality and usefulness of Xling to further enhance the existing similarity join methods and make them more practical.

F. Generalization evaluation

In this section we evaluate the generalization capability of Xling and XJoin. Another 150k dataset is sampled for each original dataset. We call the first 150k datasets used in previous experiments "the first 150k" or add "-1st" after the dataset name, while call the new dataset "the second 150k" or add "-2nd" after the name. The first and second 150k have no overlap, except NUS-WIDE.

We first evaluate the trade-off capabilities of XJoin and Xling-enhanced methods again on the second 150k. All Xlings are those trained on the first 150k and used in the previous experiments. We do not re-train them anymore for the second 150k. As shown in Figure 4, all those methods present a similar performance and trends as in Figure 3, which prove

Dataset	ϵ	Filter	FPR	FNR	#Nbrs ($\times 10^5$)	#PPQ ($\times 10^4$)	#ANPQ
FastText	0.4	LSBF	0.46	0.2908	127.73	2.05	624.58
		Xling(mean)	0.2523	0.1114	142.15	2.46	578.95
		Xling(FPR)	0.0529	0.207	141.90	2.13	664.94
	0.45	LSBF	0.5124	0.2471	296.05	2.23	1329.56
		Xling(mean)	0.3469	0.0682	325.25	2.72	1196.82
		Xling(FPR)	0.0948	0.1295	325.07	2.51	1295.92
	0.5	LSBF	0.5244	0.2062	708.64	2.37	2987.66
		Xling(mean)	0.3582	0.0376	767.57	2.87	2678.19
		Xling(FPR)	0.0802	0.099	767.23	2.68	2868.68
Word2vec	0.4	LSBF	0.4952	0.4607	3.49	1.58	22.06
		Xling(mean)	0.374	0.3197	4.63	1.78	26.08
		Xling(FPR)	0.0502	0.7028	4.20	0.68	61.81
	0.45	LSBF	0.5516	0.3977	6.77	1.78	37.98
		Xling(mean)	0.4397	0.234	8.86	2.13	41.53
		Xling(FPR)	0.0804	0.5996	8.19	1.04	78.70
	0.5	LSBF	0.5943	0.3419	13.04	1.96	66.56
		Xling(mean)	0.3019	0.2725	16.40	2.08	78.83
		Xling(FPR)	0.039	0.5882	15.15	1.15	132.22

(a) Text datasets

Dataset	ϵ	Filter	FPR	FNR	#Nbrs ($\times 10^5$)	#PPQ ($\times 10^4$)	#ANPQ
Sift	0.4	LSBF	0.2415	0.6609	4.69	0.85	54.89
		Xling(mean)	0.06	0.6119	7.18	0.62	116.65
		Xling(FPR)	0.0351	0.6771	7.08	0.49	145.37
	0.45	LSBF	0.2338	0.6358	13.04	0.96	136.42
		Xling(mean)	0.081	0.4565	19.49	1.15	170.09
		Xling(FPR)	0.0537	0.5064	19.39	1.02	190.19
	0.5	LSBF	0.2053	0.6139	35.54	1.08	330.55
		Xling(mean)	0.1798	0.237	52.83	2.02	261.36
		Xling(FPR)	0.0581	0.3678	52.50	1.63	321.55
NUS-WIDE	0.4	LSBF	0.2492	0.5687	1.39	0.76	18.19
		Xling(mean)	0.3671	0.057	2.14	1.15	18.65
		Xling(FPR)	0.0534	0.0894	2.14	0.23	94.32
	0.45	LSBF	0.2498	0.5374	2.82	0.77	36.53
		Xling(mean)	0.3302	0.071	4.06	1.05	38.56
		Xling(FPR)	0.0503	0.1123	4.06	0.24	170.27
	0.5	LSBF	0.2469	0.5029	5.15	0.78	66.42
		Xling(mean)	0.2873	0.087	6.98	0.95	73.69
		Xling(FPR)	0.0467	0.1294	6.98	0.25	276.09

(b) Image datasets

TABLE VI: The prediction quality of LSBF and Xling (mean-based or FPR-based) on different datasets, where *#Nbrs* is the total number of returned neighbors for all the queries, *#PPQ* stands for the number of *Predicted Positive Queries*, i.e., the query points predicted as positive by the filter, and *#ANPQ* presents the *Average number of Neighbors per predicted Positive Query* that equals #Nbrs over #PPQ. Due to space limit, the results on Glove and Gist datasets are hidden, which are similar to the other four.

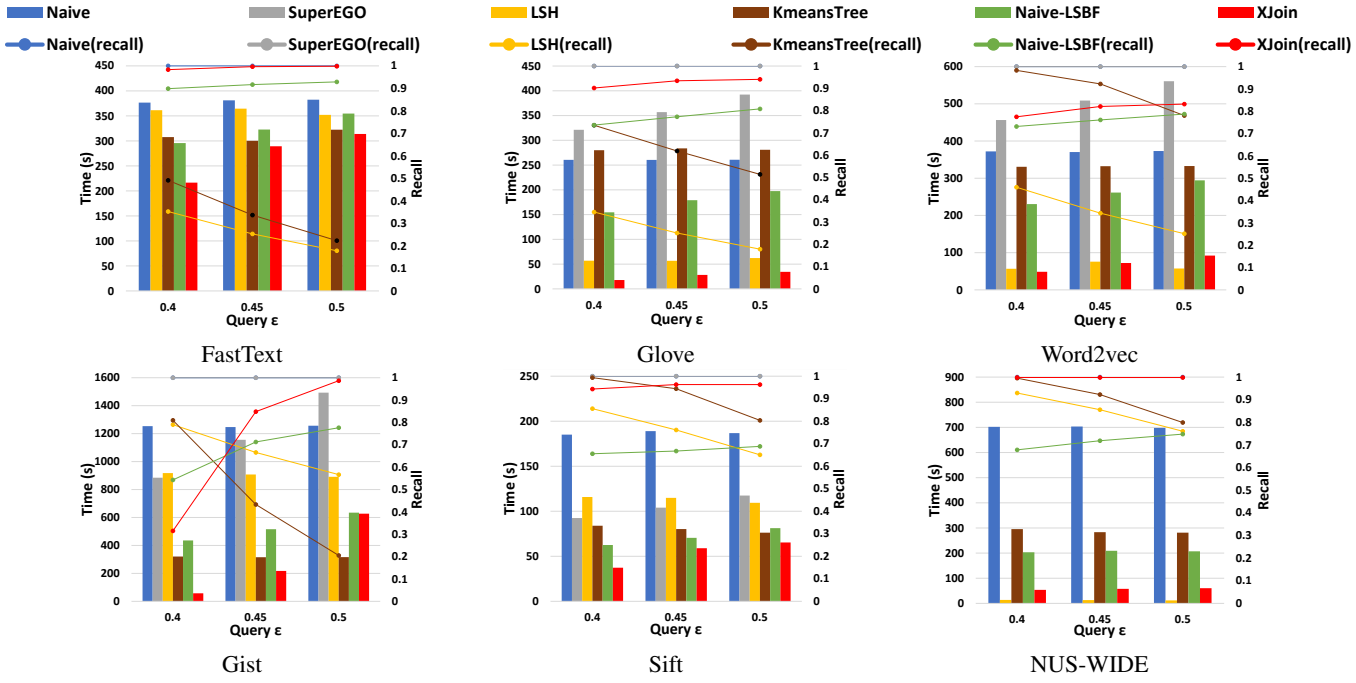


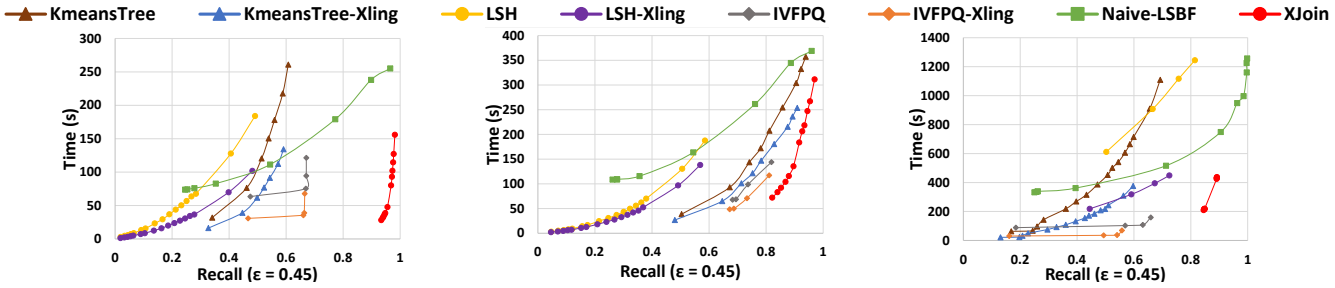
Fig. 2: End-to-end query processing time and recall for all the similarity join methods on all datasets, where the figures of FastText and NUS-WIDE do not include SuperEGO, as it cannot run on these two datasets.

our statement in the Introduction that Xling and XJoin have great generalization capability thanks to the learning model, and therefore it is not necessary to re-train Xling when the data is updated or even replaced with a fully new dataset, as long as the new data has similar distribution to the old.

To have a further quantitative view about the generalization, we also compare the speed improvement and the recall loss made by Xling when attaching it to the base similarity join methods. In Figure 5, within each method (marked as "IVFPQ", "LSH", etc.), the first bar (solid and blue, "1st time") and second bar (solid and orange, "1st Xling time") are the end-to-end running time of the original method and its Xling-enhanced version on the first 150k, while the third bar (diagonal lines and blue, "2nd time") and fourth bar (diagonal lines and orange, "2nd Xling time") are the two running time on the second 150k. The green and red lines are the percentage recall loss respectively on the first and second 150k, i.e., the difference between recall of original method and enhanced version over original recall on each dataset. The results show that neither time improvement nor recall loss has a significant difference between first and second 150k, which further prove our methods have outstanding generalization, meaning that they are practical in real world.

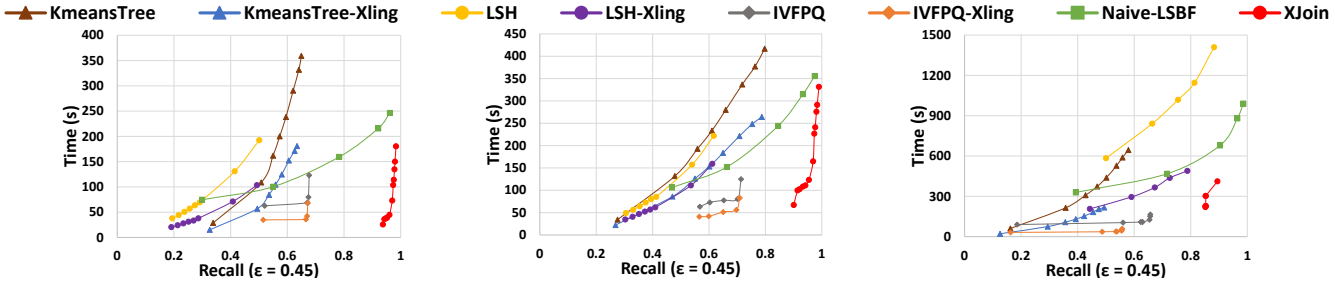
VII. Conclusion

In this paper we propose Xling, a generic framework of learned metric space Bloom filters for speeding up similarity join with quality guarantee based on machine learning. Based



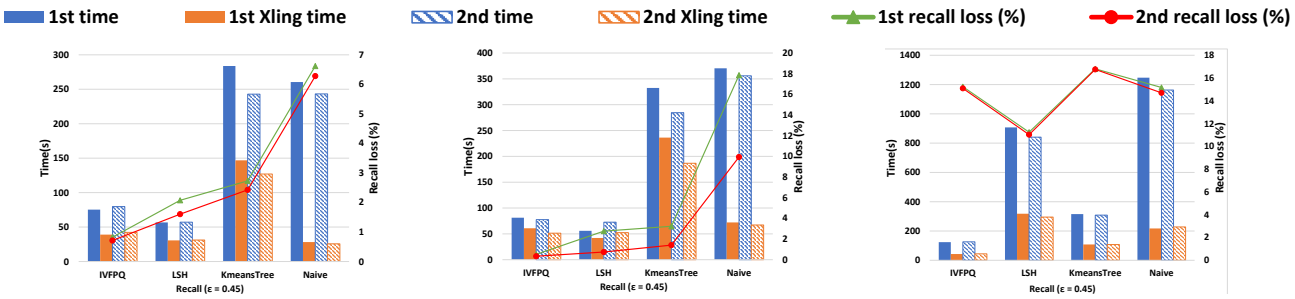
Glove Word2vec Gist

Fig. 3: Speed-quality trade-off curves for XJoin, the approximate methods and their Xling-enhanced versions on the selected datasets and ϵ, and other cases are also similar.



Glove-2nd Word2vec-2nd Gist-2nd

Fig. 4: Speed-quality trade-off curves for XJoin, the approximate methods and their Xling-enhanced versions on the second 150k datasets, where all Xlings are pre-trained on the original 150k dataset without re-training for the second



Glove 1st vs 2nd Word2vec 1st vs 2nd Gist 1st vs 2nd

Fig. 5: The differences of acceleration and recall loss resulting from Xling on the first and second 150k datasets

on Xling we develop an efficient and effective similarity join method that outperforms the state-of-the-art methods on both speed and quality, as well as having a better speed-quality trade-off capability and generalization capability. We also apply Xling onto those state-of-the-art methods to significantly further enhance them. Xling has shown the great potential in effectively speeding up a wide range of existing similarity join methods.

References

[1] B. Gyawali, L. Anastasiou, and P. Knoth, "Deduplication of scholarly documents using locality sensitive hashing and word embeddings," in *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Marseille, France: European Language Resources Association, May 2020, pp. 901–910. [Online]. Available: https://aclanthology.org/2020.lrec-1.113

[2] C. Yang, D. H. Hoang, T. Mikolov, and J. Han, "Place deduplication with embeddings," in *The World Wide Web Conference*, 2019, pp. 3420–3426.

[3] H. B. da Silva, Z. K. do Patrocínio, G. Gravier, L. Amsaleg, A. d. A. Araújo, and S. J. F. Guimaraes, "Near-duplicate video detection based on an approximate similarity self-join strategy," in *2016 14th International Workshop on Content-Based Multimedia Indexing (CBMI)*. IEEE, 2016, pp. 1–6.

[4] L. Zhou, J. Chen, A. Das, H. Min, L. Yu, M. Zhao, and J. Zou, "Serving deep learning models with deduplication from relational databases," *Proceedings of the VLDB Endowment*, vol. 15, no. 10, p. 2230–2243, Jun. 2022. [Online]. Available: http://dx.doi.org/10.14778/3547305.3547325

[5] R. Sarwar, C. Yu, N. Tungare, K. Chitavisutthivong, S. Sriratanawilai, Y. Xu, D. Chow, T. Rakthanmanon, and S. Nutanong, "An effective and scalable framework for authorship attribution query processing," *IEEE Access*, vol. 6, pp. 50 030–50 048, 2018.

[6] B. Hättasch, M. Truong-Ngoc, A. Schmidt, and C. Binnig, "It's ai match: A two-step approach for schema matching using embeddings,"

2022. [Online]. Available: https://arxiv.org/abs/2203.04366

[7] N. Adly, "Efficient record linkage using a double embedding scheme." in *DMIN*, 2009, pp. 274–281.

[8] S. Herath, M. Roughan, and G. Glonek, "Em-k indexing for approximate query matching in large-scale er," 2021.

[9] F. Nargesian, E. Zhu, K. Q. Pu, and R. J. Miller, "Table union search on open data," *Proc. VLDB Endow.*, vol. 11, no. 7, p. 813–825, mar 2018. [Online]. Available: https://doi.org/10.14778/3192965.3192973

[10] A. Berenguer, J.-N. Mazón, and D. Tomás, "Towards a tabular open data search engine for public sector information," in *2021 IEEE International Conference on Big Data (Big Data)*, 2021, pp. 5851–5853.

[11] Y. Dong, K. Takeoka, C. Xiao, and M. Oyamada, "Efficient joinable table discovery in data lakes: A high-dimensional similarity-based approach," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 456–467.

[12] X. Yuan, X. Wang, C. Wang, C. Yu, and S. Nutanong, "Privacy-preserving similarity joins over encrypted data," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 11, pp. 2763–2775, 2017.

[13] J. Yao, X. Meng, Y. Zheng, and C. Wang, "Privacy-preserving content-based similarity detection over in-the-cloud middleboxes," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1854–1870, 2023.

[14] M. Perdacher, C. Plant, and C. Böhm, "Cache-oblivious high-performance similarity join," in *Proceedings of the 2019 International Conference on Management of Data*, ser. SIGMOD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 87–104. [Online]. Available: https://doi.org/10.1145/3299869.3319859

[15] C. Böhm, B. Braunmüller, F. Krebs, and H.-P. Kriegel, "Epsilon grid order: An algorithm for the similarity join on massive high-dimensional data," in *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '01. New York, NY, USA: Association for Computing Machinery, 2001, p. 379–388. [Online]. Available: https://doi.org/10.1145/375663.375714

[16] D. V. Kalashnikov and S. Prabhakar, "Fast similarity join for multi-dimensional data," *Information Systems*, vol. 32, no. 1, pp. 160–177, 2007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306437905000761

[17] D. V. Kalashnikov, "Super-ego: fast multi-dimensional similarity join," *The VLDB Journal*, vol. 22, no. 4, pp. 561–585, 2013.

[18] C. Yu, S. Nutanong, H. Li, C. Wang, and X. Yuan, "A generic method for accelerating lsh-based similarity join processing (extended abstract)," in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, 2017, pp. 29–30.

[19] H. Li, S. Nutanong, H. Xu, c. YU, and F. Ha, "C2net: A network-efficient approach to collision counting lsh similarity join," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 3, pp. 423–436, 2019.

[20] Z. Yang, W. T. Ooi, and Q. Sun, "Hierarchical, non-uniform locality sensitive hashing and its application to video identification," in *2004 IEEE International Conference on Multimedia and Expo (ICME) (IEEE Cat. No.04TH8763)*, vol. 1, 2004, pp. 743–746 Vol.1.

[21] Y. Hua, B. Xiao, B. Veeravalli, and D. Feng, "Locality-sensitive bloom filter for approximate membership query," *IEEE Transactions on Computers*, vol. 61, no. 6, pp. 817–830, 2012.

[22] J. Qian, Q. Zhu, and H. Chen, "Integer-granularity locality-sensitive bloom filter," *IEEE Communications Letters*, vol. 20, no. 11, pp. 2125–2128, 2016.

[23] M. Goswami, R. Pagh, F. Silvestri, and J. Sivertsen, "Distance sensitive bloom filters without false negatives," 2016.

[24] J. Qian, Z. Huang, Q. Zhu, and H. Chen, "Hamming metric multi-granularity locality-sensitive bloom filter," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, p. 1660–1673, aug 2018. [Online]. Available: https://doi.org/10.1109/TNET.2018.2850536

[25] J. Qian, Q. Zhu, and H. Chen, "Multi-granularity locality-sensitive bloom filter," *IEEE Transactions on Computers*, vol. 64, no. 12, pp. 3500–3514, 2015.

[26] A. Kirsch and M. Mitzenmacher, "Distance-sensitive bloom filters," in *2006 Proceedings of the Eighth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2006, pp. 41–50.

[27] Y. Hua, X. Liu, Y. Hua, and X. Liu, "Locality-sensitive bloom filter for approximate membership query," *Searchable Storage in Cloud Computing*, pp. 99–127, 2019.

[28] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," 2018.

[29] S. Macke, A. Beutel, T. Kraska, M. Sathiamoorthy, D. Z. Cheng, and E. H. Chi, "Lifting the curse of multidimensional data with learned existence indexes," in *Workshop on ML for Systems at NeurIPS*, 2018, pp. 1–6.

[30] M. Mitzenmacher, "A model for learned bloom filters and optimizing by sandwiching," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[31] A. Bhattacharya, C. Gudesa, A. Bagchi, and S. Bedathur, "New wine in an old bottle: Data-aware hash functions for bloom filters," *Proc. VLDB Endow.*, vol. 15, no. 9, p. 1924–1936, may 2022. [Online]. Available: https://doi.org/10.14778/3538598.3538613

[32] J. Sun, G. Li, and N. Tang, "Learned cardinality estimation for similarity queries," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1745–1757.

[33] Y. Wang, C. Xiao, J. Qin, X. Cao, Y. Sun, W. Wang, and M. Onizuka, "Monotonic cardinality estimation of similarity selection: A deep learning approach," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1197–1212.

[34] J. Qin, W. Wang, C. Xiao, Y. Zhang, and Y. Wang, "High-dimensional similarity query processing for data science," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 4062–4063. [Online]. Available: https://doi.org/10.1145/3447548.3470811

[35] Y. Wang, C. Xiao, J. Qin, R. Mao, M. Onizuka, W. Wang, R. Zhang, and Y. Ishikawa, "Consistent and flexible selectivity estimation for high-dimensional data," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2319–2327.

[36] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proceedings of the 2018 international conference on management of data*, 2018, pp. 489–504.

[37] H. Zhang and Q. Zhang, "Embedjoin: Efficient edit similarity joins via embeddings," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 585–594. [Online]. Available: https://doi.org/10.1145/3097983.3098003

[38] Y. Wang and D. Z. Wang, "Learned accelerator framework for angular-distance-based high-dimensional dbscan," 2023.

[39] Y. Wang, C. Xiao, J. Qin, X. Cao, Y. Sun, W. Wang, and M. Onizuka, "Monotonic cardinality estimation of similarity selection: A deep learning approach," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1197–1212. [Online]. Available: https://doi.org/10.1145/3318464.3380570

[40] S. Macke, A. Beutel, T. Kraska, M. Sathiamoorthy, D. Z. Cheng, and E. H. Chi, "Lifting the curse of multidimensional data with learned existence indexes," 2018.

[41] Y. Wang, C. Xiao, J. Qin, R. Mao, M. Onizuka, W. Wang, R. Zhang, and Y. Ishikawa, "Consistent and flexible selectivity estimation for high-dimensional data," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2319–2327.

[42] A. Andoni, P. Indyk, T. Laarhoven, I. Razenshteyn, and L. Schmidt, "Practical and optimal lsh for angular distance," *Advances in neural information processing systems*, vol. 28, 2015.

[43] M. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration." *VISAPP (1)*, vol. 2, no. 331-340, p. 2, 2009.

[44] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 1, pp. 117–128, 2010.

[45] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *arXiv preprint arXiv:1702.08734*, 2017.

[46] Y. Wang, H. Ma, and D. Z. Wang, "Lider: An efficient high-dimensional learned index for large-scale dense passage retrieval," 2022. [Online]. Available: https://arxiv.org/abs/2205.00970

[47] L. V. Nguyen, T.-H. Nguyen, and J. J. Jung, "Content-based collaborative filtering using word embedding: A case study on movie recommendation," in *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*, ser. RACS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 96–100. [Online]. Available: https://doi.org/10.1145/3400286.3418253